



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE SOFTWARE



PROGRAMACIÓN ORIENTADA A OBJETOS

INFORME DEL PROYECTO FINAL

SISTEMA DE GESTIÓN PARA VETERINARIA

Aplicación de escritorio desarrollada en JavaFX con base de datos PostgreSQL

Asignatura	Programación Orientada a Objetos
Docente	Ing. Yadira Gissela Franco Rocha, Mgs.
Período académico	2026-A
Tipo de trabajo	Proyecto grupal
Integrantes	Shirley Tenelanda, Alisson Quiguango
Fecha de entrega	14-07-2026

Quito – Ecuador



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE SOFTWARE



Índice

1.	Introducción	3
2.	Propósito de la práctica	3
3.	Objetivo general	4
4.	Objetivos específicos	4
5.	Desarrollo	4
5.1.	Temática y problema que resuelve	4
5.2.	Arquitectura del proyecto y organización por paquetes	4
5.3.	Roles del sistema	5
5.4.	Base de datos	6
5.5.	Validaciones implementadas	6
6.	Capturas de pantalla	6
6.1.	Login	6
6.2.	Gestión de Mascotas	6
6.3.	Gestión de Propietarios	7
6.4.	Gestión de Usuarios	7
6.5.	Reportes	7
6.6.	Validaciones	8
7.	Conclusiones	9
8.	Recomendaciones	9
9.	Bibliografía	9
10.	Anexos	10
10.1	Repositorio del proyecto	10
10.2	Base de datos en PostgreSQL	10
10.3	Tablas creadas en PostgreSQL	12
10.4	Estructura del proyecto en IntelliJ IDEA	12
10.5	Tablas creadas en railway	13



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE SOFTWARE



1. Introducción

Para el proyecto final de Programación Orientada a Objetos decidimos hacer un sistema para una veterinaria. La idea es tener un programa de escritorio, hecho en JavaFX, donde se pueda registrar a los propietarios de las mascotas, las mascotas mismas, los usuarios que usan el sistema y las consultas que se les hacen. Toda esa información se guarda en una base de datos en PostgreSQL, para que no se pierda nada, aunque se cierre el programa.

Elegimos este tema porque en una veterinaria normalmente se maneja todo en papel o en Excel, y eso hace que se repitan datos, se pierdan fichas o no se sepa quién hizo qué cambio. Por eso el sistema tiene un login con tres tipos de usuario: Administrador, Cajero y Reportes, y dependiendo de cuál sea, el programa muestra unas opciones u otras. No hicimos tres pantallas distintas para cada rol, sino una sola que se adapta, lo cual nos pareció más práctico y también más ordenado a nivel de código.

En cuanto a lo que llevamos hecho, ya tenemos funcionando el login, la conexión a la base de datos, y el registro, edición, eliminación y listado (CRUD) de propietarios, mascotas y usuarios. También usamos varios conceptos de POO que vimos en clase: hicimos una clase abstracta llamada Persona de la que heredan Usuario y Propietario, y una interfaz genérica para no repetir el mismo código de guardar/actualizar/eliminar en cada módulo. El proyecto está organizado en paquetes (model, dao, controller, db, app y view) y lo hemos ido subiendo a un repositorio de Git conforme avanzamos.

En las siguientes secciones explicamos el propósito de esta práctica y el objetivo general que nos planteamos para el proyecto.

2. Propósito de la práctica

Con este proyecto buscamos resolver un problema que se da bastante seguido en negocios pequeños como una veterinaria: no tener un sistema propio para llevar el control de sus clientes, mascotas y consultas. La meta no era solo cumplir con la entrega, sino armar algo que de verdad funcione, con datos guardados en una base real y no en listas sueltas.

Nos pareció importante aplicar POO en este proyecto porque, sin usarla, hubiéramos terminado copiando y pegando el mismo código para cada pantalla y cada tabla. Al usar herencia entre Persona, Usuario y Propietario, y una interfaz para el CRUD, evitamos repetir tanto código y el proyecto queda más fácil de mantener si en el futuro se quiere agregar algo, por ejemplo, otro tipo de usuario o una nueva tabla. También nos sirvió para entender mejor, ya no solo en teoría sino programando, para qué sirve encapsular



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE SOFTWARE



los atributos de una clase o por qué conviene separar el código en paquetes según su responsabilidad (modelo, acceso a datos, controladores, etc.).

3. Objetivo general

Desarrollar un sistema de escritorio en JavaFX para gestionar propietarios, mascotas, usuarios y consultas de una veterinaria, aplicando los principios de la Programación Orientada a Objetos y usando una base de datos PostgreSQL para guardar la información y controlar el acceso mediante roles de usuario.

4. Objetivos específicos

Diseñar la arquitectura del sistema junto y el modelo entidad-relación para estructurar la información requerida por la veterinaria.

Implementar la base de datos en PostgreSQL para garantizar el almacenamiento de los datos obtenidos, controlando su acceso basado en roles

Desarrollar la interfaz gráfica en JavaFX para gestionar los módulos CRUD para propietarios, mascotas, usuarios y consultas utilizando el lenguaje Java.

Validar el correcto funcionamiento mediante pruebas de software para asegurar un buen rendimiento óptimo controlando el acceso.

5. Desarrollo

5.1. Temática y problema que resuelve

El Sistema está diseñado para optimizar procesos operativos de una clínica veterinaria. El software concentra la información en distintos módulos permitiendo la gestión manual de historiales clínicos, datos de propietarios y el control de citas para evitar que la información se duplique o incluso la pérdida de datos. El enfoque principal es garantizar un flujo de trabajo rápido y Seguro para el personal médico y administrativo.

5.2. Arquitectura del proyecto y organización por paquetes

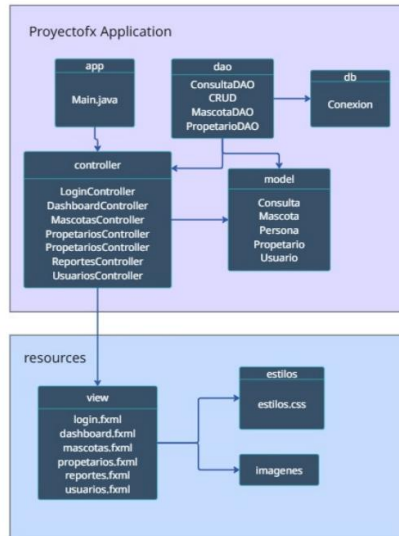
JavaFX es una plataforma de software que permite a los desarrolladores crear diversas aplicaciones cliente con gran cantidad de contenido, que funcionan de forma consistente en varias plataformas.



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE SOFTWARE



Para poder garantizar el funcionamiento y orden en el código se utilizan varios paquetes para separar el modelo de datos, la interfaz de usuario y el procesamiento de entrada del usuario. Una forma adecuada para la organización de la aplicación es separar las responsabilidades que garanticen la lógica de negocio, la interfaz de usuario y la entrada de datos mediante el uso del patrón MVC:

model: que representa los objetos del mundo real con sus atributos privados y métodos get y set. En este paquete También aplicamos la herencia y el encapsulamiento.

dao: este paquete contiene la lógica de acceso a la base de datos que se va implementando en la interfaz ICRUD.

controller: conectan los archivos fxml con la lógica. Captura los eventos de la vista y comunica al modelo las acciones que debe realizar usando el modelo CRUD

view: Es el paquete donde se va a mostrar la interfaz gráfica al usuario.

db: es el paquete que gestión la conexión a la base de datos que logra garantizar una sola instancia de la conexión.

app: es el punto de ejecución del Sistema cargando la Ventana principal.

5.3. Roles del sistema

El sistema implementa acciones y restringe, asignando un rol al usuario:

Administrador: Posee acceso completo del CRUD permitiendo la gestión de usuarios, reportes y la configuración del sistema

Cajero: Registra ventas o transacciones, consulta stock e imprime comprobantes

Reportes: este rol permite ver estadísticas, gráficos, exportar reportes sin que pueda modificar datos.

Credenciales:

- Administrador: admin@veterinaria.com / 123456
- Cajero: cajero@veterinaria.com / 123456



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE SOFTWARE



- Reportes: reportes@veterinario.com / 123456

5.4. Base de datos

La interacción entre la aplicación y el gestor de la base de datos PostgreSQL se realiza a través de un controlador. Para poder garantizar que se centralice la conexión evitando las sobrecargas de memoria al impedir la creación constante de nuevas conexiones se implementó el patrón de diseño Singleton.

5.5. Validaciones implementadas

Para evitar errores tanto al ingresar los datos y evitar fallas se implementó validaciones como:

Validaciones de campos vacíos: se evalúa que todos los campos no estén vacíos utilizando `txtNombre.getText().trim().isEmpty()`

Validaciones de tipos de datos: el usuario debe ingresar datos relacionado a los solicitado, se implementó la validación en la edad para que sea un numero positive además de numérico. El peso debe ser un numero positive y un valor numérico

Manejo de excepciones: las validaciones se manejan mediante try-catch capturando excepciones evitando que el Sistema se cierre abruptamente y mostrando el error encontrado en una Ventana.

6. Capturas de pantalla

6.1. Login



6.2. Gestión de Mascotas



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE SOFTWARE



Sistema Veterinaria - Dashboard (ADMIN)

Administrador ADMIN [Cerrar sesión](#)

MENÚ

- Mascotas
- Propietarios
- Usuarios
- Reportes

Gestión de Mascotas

Datos de la mascota

Nombre: Especie: Raza: Sexo:

Edad: Peso: Propietario:

[Guardar](#) [Editar](#) [Eliminar](#) [Limpiar](#)

Lista de mascotas

ID	Nombre	Especie	Raza	Edad	Peso	Propietario
1	Milo	Perrito	Labrador	3	24.5	Juan Perez
2	Batán	Gato	Siamés	1	1.5	Juan Perez
3	Milo	Perrito	Chihuahua	3	1.9	Juan Perez

Programación Orientada a Objetos

6.3. Gestión de Propietarios

Sistema Veterinaria - Dashboard (ADMIN)

Administrador ADMIN [Cerrar sesión](#)

MENÚ

- Mascotas
- Propietarios
- Usuarios
- Reportes

Gestión de Propietarios

Datos del propietario

Cédula: Nombre completo: Teléfono:

Dirección: Correo electrónico:

[Guardar](#) [Editar](#) [Eliminar](#) [Limpiar](#)

Lista de propietarios

ID	Cédula	Nombre	Teléfono	Dirección	Correo
1	0102000000	Juan Perez	0999999999	Quito	juan@gmail.com

Programación Orientada a Objetos

6.4. Gestión de Usuarios

Sistema Veterinaria - Dashboard (ADMIN)

Administrador ADMIN [Cerrar sesión](#)

MENÚ

- Mascotas
- Propietarios
- Usuarios
- Reportes

Gestión de Usuarios

Datos del usuario

Nombre completo: Correo electrónico: Contraseña: Rol:

[Crear usuario](#) [Editar](#) [Eliminar](#) [Limpiar](#)

Usuarios registrados

ID	Nombre	Correo	Rol
1	Administrador	admin@veterinaria.com	ADMIN
2	Cajero	cajero@veterinaria.com	CAJERO
3	Reportes	reportes@veterinaria.com	REPORTES

Programación Orientada a Objetos

6.5. Reportes



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE SOFTWARE



Sistema Veterinaria

Administrador ADMIN Centralizado

MENÚ

- Mascotas
- Propietarios
- Usuarios
- Reportes

Reportes de Consultas

Resumen general

Consultas registradas	Mascotas registradas	Ingreso total
1	3	\$25,00

Actualizar

Historial de consultas (solo lectura)

ID	Fecha	Mascota	Propietario	Diagnostico	Tratamiento	Costo	Atendido por
1	2020-07-08	Max	Juan Perez	Control general	Vitaminas	25.0	Administrador

Programación Orientada a Objetos

6.6. Validaciones

Validaciones

Campos incompletos

Debe ingresar correo y contraseña.

Aceptar

Credenciales inválidas

El correo o la contraseña son incorrectos.

Aceptar

Gestión de Mascotas

Datos de la mascota

Campos incompletos

Todos los campos son obligatorios.

Aceptar

Nombre

Edad

Años

Sexo

Seleccione

Propietario

Gestión de Mascotas

Datos de la mascota

Sin selección

Seleccione una mascota de la tabla para editar.

Aceptar

Nombre

Edad

Años

Sexo

Seleccione

Propietario

Guardar

Editar

Eliminar

Limpiar



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE SOFTWARE



7. Conclusiones

Según los resultados obtenidos, se concluye que se desarrolló de forma exitosa un sistema de escritorio funcional en JavaFX para la gestión integral de una veterinaria. La aplicación de la programación orientada a objetos garantizó un código organizado, modular y de fácil mantenimiento mientras que la conexión con PostgreSQL aseguró el resguardo de la información y un control de acceso mediante los usuarios y roles.

Se diseñó la arquitectura de software y el modelo entidad-relación, por lo que logró estructurar de forma organizada las entidades de la veterinaria y sus relaciones antes de la fase de codificación. Asimismo, la base de datos en PostgreSQL se implementó para garantizar el almacenamiento de los datos obtenidos, controlando su acceso basado en roles.

Finalmente, la implementación de una interfaz gráfica en JavaFX facilitó la interacción con los usuarios. Las pruebas realizadas permitieron comprobar el correcto funcionamiento del sistema permitiendo verificar el correcto funcionamiento de las operaciones CRUD junto con las validaciones implementadas y el acceso a las diferentes funcionalidades de acuerdo con el rol de cada usuario cumpliendo así con los objetivos planteados para el proyecto.

8. Recomendaciones

Implementar nuevas funcionalidades, el control de vacunación, la gestión de citas y recordatorios automáticos, con el fin de ampliar los servicios ofrecidos, recordatorios automáticos, la recuperación de la contraseña perdida, con el fin de poder ampliar los servicios ofrecidos por el sistema y poder mejorar la atención de la veterinaria.

Realizar mantenimiento y pruebas diaria actualizando JavaFX para contener mejoras para el rendimiento, estabilidad y seguridad de las aplicaciones JavaFX que se ejecutan en su equipo.

9. Bibliografía



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE SOFTWARE



Model-View-Controller pattern in Java: Streamlining Java web development. (s/f-a). Java Design Patterns. Recuperado el 10 de julio de 2026, de <https://java-design-patterns.com/patterns/model-view-controller/>

Preguntas frecuentes de JavaFX. (s/f). Java.com. Recuperado el 10 de julio de 2026, de <https://www.java.com/es/download/help/javafx.html>

JavaFX - architecture. (s/f). Tutorialspoint.com. Recuperado el 10 de julio de 2026, de https://www.tutorialspoint.com/javafx/javafx_architecture.htm

Códigos de colores HTML. (s/f). HTML Color Codes. Recuperado el 10 de julio de 2026, de <https://htmlcolorcodes.com/es/>

10. Anexos

10.1 Repositorio del proyecto

<https://github.com/shirleytenelanda622-ui/ProyectoFX.git>

10.2 Base de datos en PostgreSQL

```
-- TABLA USUARIOS
CREATE TABLE usuarios (
    id_usuario SERIAL PRIMARY KEY,
    nombre VARCHAR (100) NOT NULL,
    correo VARCHAR (100) UNIQUE NOT NULL,
    contrasena VARCHAR (100) NOT NULL,
    rol VARCHAR (20) NOT NULL CHECK (rol IN ('ADMIN', 'CAJERO', 'REPORTES'))
);

-- TABLA PROPIETARIOS

CREATE TABLE propietarios (
    id_propietario SERIAL PRIMARY KEY,
    cedula VARCHAR (10) UNIQUE NOT NULL,
    nombre VARCHAR (100) NOT NULL,
    telefono VARCHAR (15),
    direccion VARCHAR (150),
    correo VARCHAR (100)
);

-- TABLA MASCOTAS

CREATE TABLE mascotas (
    id_mascota SERIAL PRIMARY KEY,
    nombre VARCHAR (100) NOT NULL,
    especie VARCHAR (50) NOT NULL,
    raza VARCHAR (50),
    edad INT,
    sexo VARCHAR (10),
    peso DECIMAL (5,2),

    id_propietario INT NOT NULL,

    CONSTRAINT fk_propietario
```



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE SOFTWARE



```
        FOREIGN KEY (id_propietario)
        REFERENCES propietarios(id_propietario)
    );
-- TABLA CONSULTAS

CREATE TABLE consultas (
    id_consulta SERIAL PRIMARY KEY,

    id_mascota INT NOT NULL,
    id_usuario INT NOT NULL,

    fecha DATE NOT NULL,
    diagnostico TEXT,
    tratamiento TEXT,
    costo DECIMAL (10,2),

    CONSTRAINT fk_mascota
        FOREIGN KEY (id_mascota)
        REFERENCES mascotas(id_mascota),

    CONSTRAINT fk_usuario
        FOREIGN KEY (id_usuario)
        REFERENCES usuarios(id_usuario)
);
-- USUARIOS DE PRUEBA

INSERT INTO usuarios (nombre, correo, contrasena, rol)
VALUES
('Administrador','admin@veterinaria.com','123456','ADMIN'),
('Cajero','cajero@veterinaria.com','123456','CAJERO'),
('Reportes','reportes@veterinaria.com','123456','REPORTES');

-- PROPIETARIO DE PRUEBA

INSERT INTO propietarios (cedula,nombre,telefono,direccion,correo)
VALUES
('0102030405','Juan Pérez','0999999999','Quito','juan@gmail.com');

-- MASCOTA DE PRUEBA

INSERT INTO mascotas
(nombre,especie,raza,edad,sexo,peso,id_propietario)
VALUES
('Max','Perro','Labrador',5,'Macho',24.50,1);

-- CONSULTA DE PRUEBA

INSERT INTO consultas
(id_mascota,id_usuario,fecha,diagnóstico,tratamiento, costo)
VALUES
(1,1,CURRENT_DATE,'Control general','Vitaminas',25.00);
```



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE SOFTWARE



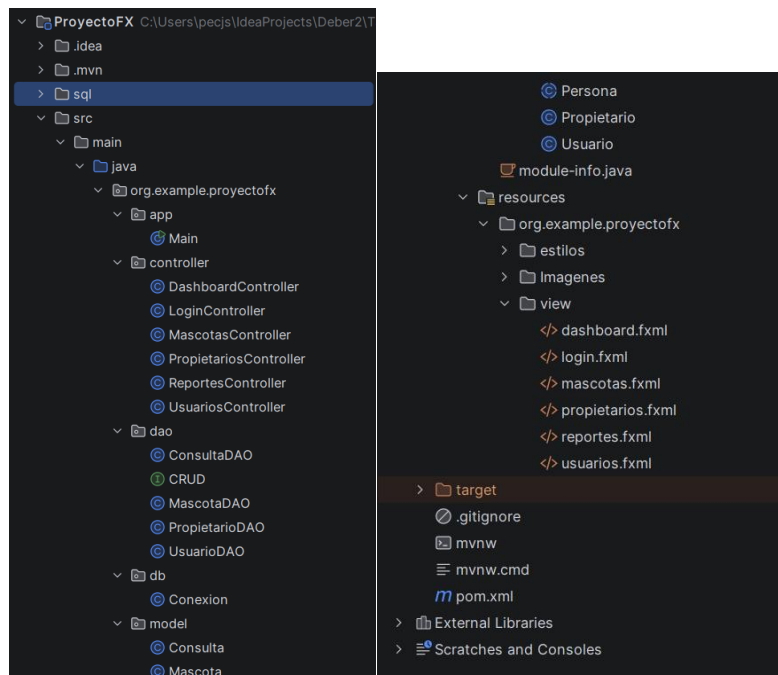
10.3 Tablas creadas en PostgreSQL

Query: `SELECT * FROM mascotas;`

Data Output

	id_mascota [PK] integer	nombre character varying (100)	especie character varying (50)	raza character varying (50)	edad integer	sexo character varying (10)	peso numeric (5,2)	id_propietario integer
1	1	Max	Perro	Labrador	5	Macho	24.50	1
2	2	Botas	Gato	Siames	1	Hembra	2.50	1
3	3	Milo	Perro	Chihuahua	3	Macho	1.90	1

10.4 Estructura del proyecto en IntelliJ IDEA.





ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE SOFTWARE



10.5 Tablas creadas en railway

